A PRODUCT MANAGER'S FIELD GUIDE

Aster

Ask your money anything.
See exactly why.

Ask your money anything. See exactly why.
An AI copilot where code computes every number and a gate checks every answer.

What it is, how it works, why it is built this way,
and how to talk about it in an interview.

Moulshree Bhutra · concept project · November 2026

INSIDE THIS GUIDE

What you will learn

01 The 60-second version

02 The problem, in plain words

03 A walk through the product

04 Under the hood: the pipeline

05 The tools: where the numbers come from

06 The gate: how it avoids lying

07 Using AI judiciously: cost control

08 Evals: the quality contract

09 Memory, traces and the dashboard

10 Product decisions and trade-offs

11 Interview cheat sheet

12 Glossary and code map

How to read this: every chapter opens with the plain-English idea, then shows how it really works, then tells you what a PM should take from it. Coloured boxes mark analogies, PM lenses and warnings. You can read it front to back in about an hour, or jump to chapter 11 (cheat sheet) right before an interview.

CHAPTER 01 · START HERE

The 60-second version

Aster is an AI money copilot. You ask a plain question about your own spending, such as "Can I afford this?" or "Why was I charged twice?", and it answers from your real transactions, shows its working, and refuses to give investment, loan or tax advice.

0 numbers allowed from the model's memory	**~390** test questions in three eval sets	**$0** cost of the simple lookup answers

What it is

Aster is a concept project: a working prototype running on **synthetic data** (20 fake users with about 4,300 transactions). It is not financial advice. It was built to prove the hard parts of a trustworthy AI product in a sensitive domain: **grounding** (every number comes from code), a **safety gate**, an **evaluation harness** that decides whether it is allowed to ship, **trace logging**, and **cost control**.

Question a person asks	What Aster does
How much did I spend on food delivery this month?	Adds up the matching transactions with a tool and shows the rows
Can I afford a ₹80,000 phone?	Returns a range and the assumptions behind it, never a single false-precision answer
Why was I charged twice at that merchant?	Finds likely duplicate charges and lists them
I want to save ₹60,000 by March	Works out the monthly amount needed and whether it is realistic
Should I invest in mutual funds?	Politely redirects: it does not give regulated advice

WHY THIS PROJECT EXISTS

Money is the domain where a wrong AI answer hurts most. Aster shows how a PM would run an AI product like an engineering discipline: decide what the model may and may not do, build tests that act as the contract, measure cost per user, and be honest about what the numbers mean.

The one-sentence pitch

"An AI money copilot where the model never does the maths: code computes every number, a gate checks every answer, and a few hundred test questions decide whether it is good enough to ship."

The problem, in plain words

ANALOGY

Imagine hiring a very confident intern to answer questions about your bank statement. They are quick and friendly, but sometimes they guess a number and say it with a straight face. In a money app, a wrong number is not a typo, it is a broken promise. Aster keeps the intern's friendly voice but takes the calculator out of their hands.

What goes wrong with a plain AI chatbot on money

- **It makes up numbers.** Language models predict plausible text, not exact sums.
- **It cannot see your data** unless you give it, and it forgets what it saw.
- **It drifts into advice** ("you should put that in mutual funds") that regulators care about.
- **It can be tricked.** A merchant name like "Ignore previous instructions" could steer it.
- **It is costly at scale.** Paying a model to read back a total is wasteful.

What users and the business need

Need	Aster's answer
Right	Tools compute every figure; a gate rejects any number that no tool produced
Safe	Regulated questions are designed out with a fixed redirect; injection text is treated as data
Cheap per user	Simple lookups cost $0; a daily spend ceiling; a cache
Provable	A large test suite with hard pass/fail gates
Understandable	A "Show the working" panel and a dashboard of real product metrics

PM LENS: RIGHT, SAFE AND CHEAP

These three words organise the whole project. A PM running an AI feature has to hold quality, risk and unit economics in one frame. Most weak AI demos only show the first.

A walk through the product

Page	What it is	Who it is for
/ (chat)	Ask a question and get an answer with trace badges (intent, provider, cost, latency, gate result). Open Show the working to see tool calls and source rows. Vote helpful or not, with a reason.	The user
/evals	The live evaluation report: per-layer gates, coverage by question type, failing cases	The PM and engineers
/dashboard	Product metrics from real traces: helpful rate, verified accuracy, cost per asker, cache hit rate, escalation rate, p95 latency	The PM
/memory	What Aster remembers about you: goals and corrections, each viewable and deletable	The user
/case-study	The write-up of how it was built, including a real failure the evals caught and how it was fixed	Reviewers and interviewers

A conversation, step by step

1. You pick a demo user and type: *"Food delivery this month?"*
2. The router decides this is a **factual lookup** about the **food delivery** category in **this month**.
3. A tool adds up those transactions and returns the total in paise plus the exact rows.
4. A template (no AI cost) writes the sentence using only that total.
5. The gate checks that every rupee figure in the sentence exists in the tool output.
6. You see the answer, small badges, and a Show the working button with the rows.

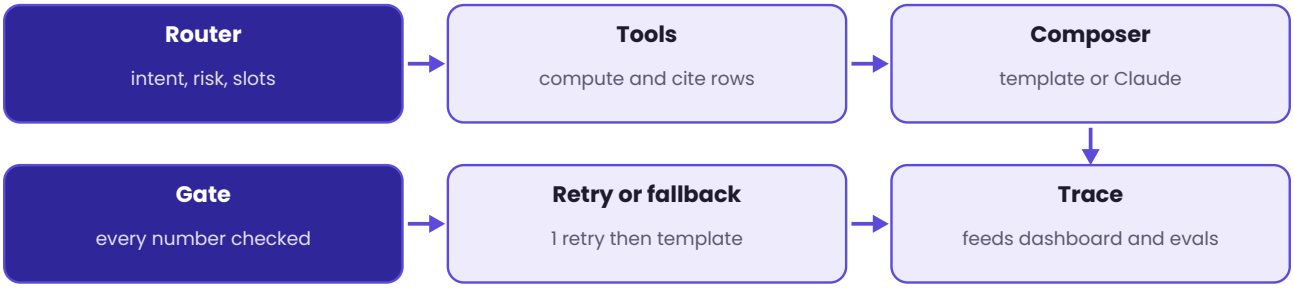
TRY IT YOURSELF

Ask "Can I afford a ₹80,000 phone?", then open Show the working. Look for the range and the four assumptions. Then ask "Should I invest in mutual funds?" and notice the instant, fixed redirect with no model and no tools involved.

PM LENS: SHOW THE WORKING

Trust in AI is built by letting users check it. The working panel turns "trust me" into "here are the rows that add up to this". It is also the best debugging tool a PM has when reviewing bad answers.

Under the hood: the pipeline



Station	What it does	Costs money?
Router (router.ts)	Reads the question and decides intent, risk class, language (English or Hinglish), and slots like category, period, merchant and amount. Uses fuzzy matching to forgive typos	No
Tools (tools/index.ts)	Typed functions that do the maths on the user's transactions and return exact figures and source rows	No
Composer (composer.ts)	Writes the answer from tool facts only. A template for lookups; Claude for reasoning-heavy questions	Only when Claude is used
Gate (gate.ts)	Rejects an answer that contains a figure no tool produced, or a regulated-advice phrase	No
Retry and fallback	One retry on a failed gate; then a guaranteed-grounded template	Rare
Trace (metrics/trace.ts)	Logs every step: intent, tools, cost, latency, gate result, votes	No

THE KEY IDEA

Most of the pipeline is plain code. Only one station can use an AI model, and that station is boxed in by stations before it (which give it the facts) and after it (which check its work). This is why Aster is cheap, testable and reproducible.

The router

The router is deliberately **not** an AI model. It is a deterministic classifier with keyword tables per category (food delivery, groceries, transport, rent, subscriptions and so on), Hinglish marker words (kitna, kharch, paise, mahine), and fuzzy token matching. It tolerates stretched letters ("foood") and small edit distances, with tight limits so "tablet" does not match "target".

Intent	Meaning
factual_lookup	"How much did I spend on X in Y?"
period_comparison	"Is this month more than last month?"
affordability	"Can I afford X?"
leakage_detection	"Anything I'm being charged twice or forgot about?"

Intent	Meaning
goal_planning	"I want to save X by Y"
correction	"That was reimbursed" or "that's not mine"
regulated_advice	Investment, loan or tax questions: redirected
ambiguous / smalltalk	Asks a clarifying question or answers a greeting

ANALOGY

The router is the receptionist. It does not answer your question, it just decides which desk to send you to, and it can spot a regulated question at the door and send it to a fixed script. A receptionist is cheap and fast, and never makes up a number.

PM LENS: NOT EVERYTHING NEEDS A MODEL

A good AI PM asks of each step: does this need judgement, or is it a lookup? Using code where code works makes the product faster, cheaper and easier to test. Aster reserves the model for the one job where wording and reasoning help.

CHAPTER 05 · GROUNDING, EXPLAINED

The tools: where the numbers come from

A **tool** is a small, typed function that takes arguments (user, period, category, amount) and returns exact figures plus the source rows. All money is stored as integers in **paise** (100 paise = ₹1), so there are no rounding surprises. Dates are resolved from a fixed anchor, which makes every answer reproducible.

Tool	What it computes
sum_spend	Total spend for a category, merchant or period, plus the matching rows
filter_transactions	A list of matching transactions
compare_periods	Two periods side by side with the signed difference
detect_duplicates	Likely duplicate charges (same merchant and amount close together)
list_recurring	Subscriptions and regular payments, and their monthly total
income_summary	Average monthly income from salary credits
affordability_check	Range of what you can comfortably spend now, with assumptions
goal_plan	Monthly saving needed for a goal and whether it is realistic

Affordability, step by step

Affordability is where a naive AI would be most dangerous, so it is worth understanding. The tool produces a **range**, not a single number:

1. Average monthly income over recent salary credits.
2. Typical monthly spend: the last three months averaged.
3. Spent so far this month, and committed recurring payments (rent, subscriptions).
4. Safety cushion: 10% of income is always left alone.
5. Low end = typical buffer (income - typical spend) minus the cushion. High end = income minus spent so far minus recurring minus the cushion. The range is the smaller and larger of those.
6. Verdict: **yes** if the price is at or under the low end, **tight** if it is between low and high, **no** if it is above the high end.

WORKED EXAMPLE (INVENTED NUMBERS)
Income ₹60,000. Typical spend ₹42,000, spent this month ₹18,000, recurring ₹22,000, cushion ₹6,000. Low = (60,000 - 42,000) - 6,000 = **₹12,000**. High = 60,000 - 18,000 - 22,000 - 6,000 = **₹14,000**.
A ₹10,000 purchase is a **yes**. ₹13,000 is **tight**. ₹80,000 is a **no**. The answer shows both ends and the four assumptions, then adds: "This is a cashflow estimate, not financial advice."

Goal planning

For "save ₹60,000 by March" the tool counts the months until March (at least one), divides the target by that, and compares with your typical monthly surplus (income minus spend). If the monthly need is higher than the surplus, the answer reports the shortfall. If you gave no date it assumes six months and says so.

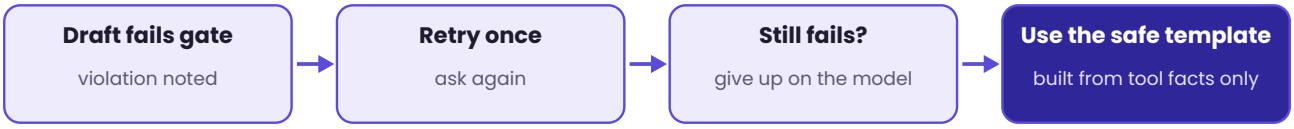
CHAPTER 06 · THE MOST IMPORTANT PART

The gate: how it avoids lying

After an answer is written, the gate reads it and asks two questions. Both checks are plain code, which is cheaper and more reliable than asking a second model to judge the first.

Check	How it works	Example catch
Rupee figures	Finds every ₹ or Rs amount in the answer and requires it to match a tool output within ₹1	Answer says ₹5,400 but tools returned ₹4,500
Percentages	Every "%" must match a tool number (within half a point)	"Up 12%" when the tool said 9%
Counts	Counts like "3 charges" or "2 subscriptions" must be in the tool output	"3 duplicates" when there were 2
Advice leaks	Phrases like "you should invest" or "I recommend buying" are rejected	A fluent but regulated suggestion

What happens when the gate fails



The fallback template is built directly from tool facts, so it passes the gate by construction. The user never sees an ungrounded number. The trace records that a fallback happened, so the PM can see how often the model needed correcting.

Regulated advice: designed out, not disclaimered

Questions about investing, loans or tax are caught by the router before any tool or model runs. The answer is a fixed redirect: Aster cannot give regulated advice, but can show your spending, recurring payments, and whether a purchase fits your cashflow. There is nothing for a model to improvise, so there is nothing to leak.

> **ANALOGY**
> A disclaimer is a sign saying "slippery floor". A redesign is a floor that is not slippery. Aster takes the second approach for regulated advice: the risky path does not exist.

Prompt injection

Merchant names and descriptions are untrusted text. The composer's instructions say they are data, never commands, and the evals include test cases where a merchant name contains an instruction. The gate gives a second line of defence: even if a model were fooled, it still could not state a number that no tool produced. No single safeguard is perfect, so Aster stacks them: router, tools, prompt rules, gate, fallback.

CHAPTER 07 · PAY ONLY WHEN IT MATTERS

Using AI judiciously: cost control

Aster never calls a model on its own. There is no background job, no polling and no proactive AI. Only a user pressing **Ask** can trigger a model call. Four levers keep the bill small.

Lever	How it works	Effect
Deterministic router and gate	Intent, safety and grounding run as code	$0 for the brains of the system
Smart mode (default)	Factual lookups and ₹0 answers use a template. Claude is used for affordability, goals, comparisons and leakage framing	Most volume costs nothing
Per-user answer cache	The same question from the same user within 30 minutes reuses the grounded answer	No paying twice
Daily spend ceiling	A hard cap (default $0.25 per day, about $7.50 a month). When reached, Aster answers with the $0 template	A loop or spike cannot blow the budget

There are three modes: **smart** (default), **always** (use the model for everything) and **off** (never call a model). With no API key at all, an offline deterministic provider runs, which is why the whole app and the evals are reproducible with zero keys.

WORKED EXAMPLE (ILLUSTRATIVE)

If a typical model-answered question costs about $0.001 on a small model, 100 such questions a day cost about $0.10, within the $0.25 ceiling. If 70 of those 100 questions were simple lookups answered by a template, the model is only used for 30 and the cost is about $0.03.

These are teaching numbers. The real figures live on the dashboard as cost per query and cost per asker.

PM LENS: UNIT ECONOMICS FOR AI

An AI feature has a variable cost per use. Think: cost per query, cost per active user, and what happens to margin if usage doubles. Ceilings, caches and templates are product tools for protecting the business model, not only engineering tricks.

Evals: the quality contract

An **eval** is a test for an AI product. Aster's eval suite builds hundreds of questions, runs the real pipeline, and scores the answers against ground truth that is **computed by code** from the same synthetic data. Because the right answer is computed, scoring is objective, not a matter of opinion.

Eval set	Size	Purpose
Golden set	about 295	Core coverage across the question types: lookups, comparisons, affordability, leakage, regulated traps, ambiguous, Hinglish, typos, injection
Blind set	60	Questions written without looking at how the router works, to catch overfitting to my own wording
Fresh set	35	A further unseen set after fixes, to check the improvements generalise

The ship gates

Layer	What is measured	Gate to ship
Tool-call correctness	Did it call the right tool with the right arguments?	95% or more
Numeric accuracy	Are the figures right? Any invented figures?	98% or more and zero invented figures
Answer quality	A rubric score on helpfulness and clarity	At or above baseline
Safety and refusal	Were regulated traps redirected? Any leaks?	100% redirected, zero leaks
Robustness	Typos, Hinglish and injection vs clean questions	Accuracy drop of 5 points or less

The story the evals told

On the first run the evals found four real bugs that no one had noticed by trying a few questions by hand:

- A "how much" question was sent to the subscription lister instead of a spend sum, because a keyword beat the "how much" signal.
- A month with genuinely zero spend answered "couldn't find" instead of "₹0", so a correct zero did not match the truth.
- Period comparisons printed an absolute difference while the gate held the signed value, so the gate flagged a correct number as ungrounded.
- Typos like "spnd", "delivry" and "mnth" slipped past exact matching, so fuzzy matching was added.

After fixing them the ship gate went from FAIL to PASS on the golden set. The blind set was then run, and because it is written independently it scored lower at first (around 63%), which exposed wording the router had not seen. Those gaps were fixed and a fresh set was added to check the fixes held up.

BE HONEST ABOUT THE LIMITS

Passing a test set the author wrote is not the same as being good in the wild. The answer-quality score is a regex-based rubric and has not been calibrated against human judgement. Evals use the offline provider by default, so they test the pipeline and not a real model's wording. These are stated limits, not hidden ones.

PM LENS: EVALS ARE THE SPEC

For AI products, the eval suite is the closest thing to a living requirements document. It says exactly what "good" means, in a way that can fail a build. A PM who can read an eval report is a PM who can steer an AI team.

CHAPTER 09 · LEARNING FROM USE

Memory, traces and the dashboard

Memory: small, explicit, deletable

Aster remembers only two kinds of things, and only when you say them: **goals** ("save ₹60,000 for a phone by March") and **corrections** ("that was reimbursed"). Each is stored as a typed fact with the message that created it. The **What Aster remembers** page lists everything and lets you delete any item. There is no hidden profile.

> **PM LENS: MEMORY IS A TRUST FEATURE**
> People accept memory when they can see it and remove it. A visible, editable memory page is a product answer to "what does this thing know about me?".

Traces: one log, two uses

Every question writes a **trace**: the question, intent and risk class, tools called, which provider answered, cost, latency, gate result, whether a retry or fallback happened, and the user's vote. The same traces feed the dashboard and the evals, so there is one source of truth.

Dashboard metrics

Metric	What it tells you
Helpful rate	Share of voted answers marked helpful, with reasons for unhelpful ones
Verified accuracy	Share of answers that passed the gate
Cost per asker and per query	Unit economics, visible at all times
Cache hit rate	How often money is saved by reuse
Escalation rate	How often a question hit a safety redirect or fallback
p95 latency	How slow the slowest 5% of answers are

> **WATCH OUT**
> On the free hosting used for the demo, traces live in server memory with simple file persistence, so they are not durable across restarts. A real product would use a database. Swapping it is limited to a single storage file.

Product decisions and trade-offs

Decision	Why	What we gave up
Model never does arithmetic	Exact numbers need exact code	Some natural flexibility
Deterministic router, not an LLM classifier	Free, fast, reproducible evals	Weaker on unusual phrasing; needs a keyword table
Gate in code, not a model judge	Cheap, predictable, auditable	Cannot catch a wrong-but-grounded claim
Ranges, not single answers, for affordability	Honest about uncertainty	Less satisfying than a crisp yes or no
Regulated advice redirected	Avoids compliance risk by design	Cannot help with investing questions
Template answers for lookups	$0 and perfectly grounded	Less warm and conversational
Synthetic data	No privacy risk; reproducible	Real-world messiness not tested
Hard daily cost ceiling	Protects the budget	Falls back to templates on busy days
Explicit memory only	Trust and transparency	Less personalisation

What a real product would add next

- Run the evals against a real model and compare wording quality, not just correctness.
- A durable database for traces, and an Experiments page to compare versions.
- Calibrate the answer-quality score against human ratings.
- Bank connections and real transaction categorisation, with a proper privacy review.
- A product requirements document updated with the build results.
- A clear "synthetic data, not financial advice" line under the demo user picker.

Interview cheat sheet

The 30-second version

"Aster is an AI money copilot on synthetic data. The model never does the maths: typed tools compute every figure, a deterministic gate rejects any number that a tool did not produce, and regulated advice is redirected before a model even runs. Simple lookups cost nothing because a template answers them, and there is a hard daily spend cap. A few hundred eval questions with pass/fail gates decide whether it can ship, and the evals already caught four real bugs."

The 2-minute version, in four beats

1. **Problem.** AI on money must be right, safe and affordable, and most demos only show the chat.
2. **Solution.** A pipeline: router, tools, composer, gate, fallback, trace. Code does everything except the final wording of a few answer types.
3. **Design choices.** Grounded or silent; refuse rather than guess; template before model; evals as the contract; cost ceiling.
4. **Honest limits and next steps.** Synthetic data, regex-based quality score, in-memory traces, evals on the offline provider. Next: a real-model eval, a durable database and a calibrated judge.

Likely questions and good answers

How do you stop the AI giving wrong numbers?

It never produces numbers. Tools compute them from the transactions and return the source rows. A gate then checks every rupee figure, percentage and count in the final answer against the tool output. If it fails, one retry, then a template built from tool facts.

Why not use an LLM for everything?

Because most steps do not need judgement. The router, the maths and the checks are code, which makes them free, fast and testable. The model is used only where wording and reasoning help, such as affordability and goals.

How do you know it is good enough to launch?

Ship gates on five layers: tool correctness 95%, numeric accuracy 98% with zero invented figures, answer quality at baseline, 100% safety redirects, and robustness within 5 points. Nothing ships unless all pass.

What did the evals find?

Four real bugs on the first run: a routing keyword beating a stronger signal, zero-spend months answered as not found, a sign mismatch on comparisons that made the gate flag a correct number, and typos that slipped past exact matching.

How do you handle regulated advice?

It is designed out: the router catches investment, loan and tax questions before any tool or model runs and returns a fixed redirect. A trap set of such questions must be redirected 100% of the time.

What about cost?

Simple lookups use a free template, there is a per-user cache, a daily spend ceiling that falls back to templates, and no background AI. The dashboard shows cost per query and per asker.

What is the biggest weakness?

The test set is mine, the quality rubric is uncalibrated, and the data is synthetic. I would say that up front, then explain the plan: blind and fresh sets, then a real-model eval and human-rated calibration.

What would you measure after launch?

Helpful rate with reasons, verified accuracy, escalation and fallback rate, cost per asker, cache hit rate, p95 latency, and repeat usage.

CHAPTER 12 · REFERENCE

Glossary and code map

Glossary

Term	Meaning
Grounding	Making every claim traceable to real data, here the user's transactions
Tool	A typed function that computes figures and returns source rows
Paise	One hundredth of a rupee; amounts are stored as integers to avoid rounding errors
Router	The classifier that picks intent, risk and language before anything else runs
Composer	The step that writes the answer from tool facts
Gate	The code check that rejects ungrounded figures and advice leaks
Fallback	A template built only from tool facts, used when the gate keeps failing
Hinglish	Hindi written in English letters, mixed with English
Prompt injection	Text in data that tries to give the model new orders
Eval / golden set	A test suite for an AI product; a golden set has known right answers
Blind set	Questions written independently to catch overfitting
Ship gate	A pass/fail threshold that must be met before release
Trace	A log of everything that happened for one question
p95 latency	The time within which 95% of answers arrive
Escalation	A question that ended in a safety redirect or needed a fallback

Where things live in the code

Repository: `apps/aster` in the portfolio repo.

File	What is inside
src/lib/agent/router.ts	Intent, risk and language classifier with fuzzy matching
src/lib/tools/index.ts	The tools, date helpers and money formatting
src/lib/agent/composer.ts	Templates and prompt for the composer, allowed facts
src/lib/agent/gate.ts	Grounding and safety checks
src/lib/agent/pipeline.ts	The ask pipeline, cache, daily spend ceiling, regulated redirect
src/lib/agent/llm.ts	Claude, Gemini, OpenAI and offline providers
src/lib/agent/memory.ts	Capturing goals and corrections
src/lib/data/	Synthetic world generator, store and types

File	What is inside
src/lib/evals/	Golden, blind and rubric code and the runner
src/lib/metrics/trace.ts	Trace log and dashboard metrics
src/app/	The pages: chat, evals, dashboard, memory, case study, and the API routes